

Managing software environments

apt, venv, uv, Poetry, conda / mamba / micromamba, and Jupyter kernels, with the learning containers as the worked example. Written 2026-10-02. Commands checked with uv 0.11, Poetry 2.4, micromamba 2.9 and ipykernel 7.4.

The short version. An *environment* is a folder holding an interpreter and a set of installed packages. Every tool here does three things: it **creates** that folder, **records** what should be in it (an intent file you edit, plus a lock file the tool writes), and **rebuilds** it exactly from the lock. They differ in *what* they can install (Python only, or anything) and in *how you refer to* an environment (by name, or by the project folder it belongs to).

1. Why environments are hard: native code

2. Two lists: the intent file and the lock file

3. Two ways to refer to an environment

4. System packages (apt) and plain venv

5. uv

6. Poetry
7. conda, mamba and micromamba

8. Other tools: pixi, Spack, Nix

9. Jupyter kernels: one front end, many environments

10. Worked example: the thesis image

11. Which tool when, and common pitfalls

12. Quick reference

1 · Why environments are hard: native code

Many Python packages are thin wrappers around compiled C, C++ or Fortran code, and that code depends on system libraries (BLAS, Boost, LLVM, GDAL...). Compiled code is also tied to the **ABI** (the binary interface) of what it was built against: a graph-tool built against numpy 1.26 crashes on import with numpy 2 ("compiled using NumPy 1.x cannot be run in NumPy 2.x"). There are three ways packages reach you, and they handle native code differently:

Source	How native code is handled	Consequence
apt (Ubuntu)	Every library is a shared system package; all packages are built together against the same versions.	Consistent and well tested, but one version per OS , often one or two years old. Installs need root.
PyPI wheels (pip, uv, Poetry)	Each wheel bundles its own copy of the native libraries it needs ("manylinux" wheels).	Works well for numpy, pandas, scipy. Cannot express "I need Boost" or "I need GAP", so packages like graph-tool or Sage are not on PyPI at all.
conda-forge (conda, mamba, micromamba, pixi)	Every library (libboost, openblas, gdal, R, compilers) is its own package; a solver keeps the whole environment ABI-consistent.	Like apt, but per environment, any version, any language, no root. Environments are large (often GBs).

What an environment physically is

- venv** (made by `python -m venv`, `uv` or `Poetry`): a folder with `bin/python` (a link to an existing interpreter), `bin/pip`, `bin/activate` and `lib/python3.X/site-packages/`. It only holds Python packages.
- conda prefix** (`conda`, `micromamba`, `pixi`): a whole mini-Linux userland: `bin/` (its own python, R, gcc, tools), `lib/` (shared libraries), `include/`, `share/`.

- **Activating** an environment only changes your shell: it puts `<env>/bin` first on `PATH` and sets a variable (`VIRTUAL_ENV` or `CONDA_PREFIX`). Calling `<env>/bin/python` directly gives the same result without activating, which is what scripts and Dockerfiles do.

2 · Two lists: the intent file and the lock file

Every serious workflow keeps two lists:

1. **Intent file**: what *you* asked for, with loose constraints (`pandas>=2`, `graph-tool`). You edit it.
2. **Lock file**: what the solver *picked*: every package, including indirect dependencies, at exact versions, usually with hashes. Generated, never edited by hand. It is what makes a rebuild next month identical.

Tool	Intent file	Lock file	Rebuild exactly
pip + venv	— (or <code>requirements.in</code>)	<code>pip freeze > requirements.txt</code>	<code>pip install -r requirements.txt</code>
uv, pip mode	<code>requirements.in</code>	<code>requirements.txt</code> from <code>uv pip compile</code>	<code>uv pip sync requirements.txt</code>
uv, project mode	<code>pyproject.toml</code>	<code>uv.lock</code>	<code>uv sync --locked</code>
Poetry	<code>pyproject.toml</code>	<code>poetry.lock</code>	<code>poetry install / poetry sync</code>
conda / micromamba	<code>environment.yml</code>	<code>--explicit export</code> , or <code>conda-lock.yml</code>	<code>micromamba create -f <lock></code>
pixi	<code>pixi.toml</code> / <code>pyproject.toml</code>	<code>pixi.lock</code>	<code>pixi install --locked</code>

Why pip freeze alone is not enough. It gives you a lock without an intent. Six months later you cannot tell the 5 packages you wanted from the 60 that came along, so you cannot upgrade sensibly. The rule everywhere: **edit the intent file → re-lock → commit both to git → build from the lock.**

3 · Two ways to refer to an environment

conda: named, standalone environments

An environment is a thing in its own right, stored centrally, and you activate it by name from any folder. One environment can serve many projects.

```
~/micromamba/envs/
├── stats/ ← micromamba activate stats
└── gis/    (from anywhere)
```

You can also give a **path** instead of a name (`-p /opt/sage`); the Dockerfiles do this.

uv / Poetry: one environment per project folder

The environment belongs to a project. The folder's `pyproject.toml` defines it and its `.venv` is the environment. The "name" is the folder.

```
~/projects/
├── stats/ pyproject.toml uv.lock .venv/
└── gis/   pyproject.toml uv.lock .venv/
```

You don't activate by name: you `cd` into the folder and use `uv run ... / poetry run ...`, which find the environment from where you are.

The folder model is a deliberate choice: the environment travels with the code and its lock file is committed next to it. It avoids the classic conda problem of a shared `stats` environment that five projects quietly depend on and that drifts over time. The price is that "a scratch environment I use from anywhere" needs a small workaround (section 5).

4 · System packages (apt) and plain venv

The baseline. `apt` gives you a consistent, tested stack for the whole OS; `python -m venv` gives you an isolated place for PyPI packages on top of an interpreter.

```
sudo apt install python3-venv python3-graph-tool
python3 -m venv ~/.venvs/work                # create
source ~/.venvs/work/bin/activate            # activate (optional)
pip install pandas                           # add packages
pip freeze > requirements.txt                 # snapshot
deactivate; rm -rf ~/.venvs/work              # delete = remove the folder
```

Mixing apt and a venv: `--system-site-packages`

A normal venv ignores everything apt installed. With `--system-site-packages` it *also* sees the system's Python packages, so you can use apt's graph-tool and add PyPI extras without touching the system Python. The danger is that a later `pip install` upgrades numpy inside the venv and breaks the apt package compiled against the old one. The fix used in the thesis image is a **constraints file**:

```
python3 -m venv --system-site-packages /opt/thesis
/opt/thesis/bin/pip freeze | grep -E '^(numpy|scipy|pandas)==' > /opt/thesis/constraints.txt
printf '[install]\nconstraint = /opt/thesis/constraints.txt\n' > /opt/thesis/pip.conf
# pip reads $VIRTUAL_ENV/pip.conf: every later `pip install x` must keep numpy==1.26.x
```

A constraint is not a request: it does not install anything; it only forbids other versions if something else pulls the package in.

5 · uv

A single Rust program from Astral (the makers of ruff) that replaces pip, venv, pip-tools, pyenv and pipx. It is 10–100× faster than pip thanks to caching and parallel downloads, writes cross-platform lock files, and can download Python versions itself. It installs **only Python packages** from PyPI.

Project mode (pyproject.toml + uv.lock + .venv)

```
uv init                                     # new project; `uv init --bare` = only pyproject.toml
uv add pandas "numpy<2"                    # edits pyproject.toml, re-locks, installs into .venv
uv add --dev pytest ruff                   # dependency group "dev"
uv add --group docs mkdocs                  # any named group
uv remove pandas
uv lock                                     # re-resolve after editing pyproject.toml by hand
uv lock --upgrade-package numpy              # move one package to the newest allowed version
uv sync                                     # make .venv match the lock exactly (removes extras)
uv sync --locked                             # fail if the lock is out of date (CI, Docker)
uv sync --no-dev --group docs                # choose groups
uv tree                                     # dependency tree
uv run python analysis.py                    # sync, then run inside .venv; no activation needed
uv export --format requirements-txt > requirements.txt # for tools that want pip format
```

pip mode (no project; any venv)

Good for a container's shared venv such as `/opt/venv`, or for scripts that are not packages.

```
uv venv /opt/venv --python 3.12
# requirements.in ← you write it:  jupyterlab  pandas  numpy<2
uv pip compile requirements.in -o requirements.txt # the lock: every package pinned
uv pip sync --python /opt/venv requirements.txt   # install exactly that, remove the rest
uv pip install --python /opt/venv sympy           # quick ad-hoc add
uv pip freeze --python /opt/venv
```

Without `--python`, `uv pip` uses the activated venv, or a `.venv` in the current folder.

Several environments with uv

Option A, one folder per environment (locked). The folder is the name; `--project` lets you use it from anywhere.

```
mkdir -p ~/envs/stats && cd ~/envs/stats && uv init --bare && uv add numpy scipy statsmodels
mkdir -p ~/envs/gis && cd ~/envs/gis && uv init --bare && uv add geopandas rasterio
uv run --project ~/envs/stats python analysis.py
uv run --project ~/envs/gis ipython
```

Option B, plain venvs in one place (closest to conda). The path is the name; add a shell function to activate by name.

```
uv venv ~/.venvs/stats --python 3.12
uv venv ~/.venvs/gis --python 3.13
uv pip install --python ~/.venvs/stats numpy scipy
workon() { source ~/.venvs/"$1"/bin/activate; } # in ~/.bashrc; usage: workon gis
```

Option B has no lock file unless you add the `requirements.in` → `uv pip compile` step.

Python versions and Docker

```
uv python install 3.13 # downloads a standalone CPython
uv python list
uv sync --python 3.12 # rebuild .venv on another interpreter
uvx ruff check . # run a tool in a throw-away environment
UV_PROJECT_ENVIRONMENT=/opt/venv uv sync --locked # Docker: put the project env at a fixed path
```

Watch out. `uv init` writes `requires-python = ">=<newest Python it found>"`. If you later want an older interpreter, lower that line first, or `uv` (and `Poetry`) will refuse.

6 · Poetry

The older project manager (since 2018) that `uv`'s project mode resembles. Same idea: `pyproject.toml` as intent, `poetry.lock` as lock, one venv per project, plus `poetry build/publish` for packaging. Slower than `uv`, does not install Python versions, and needs its own Python to run (which is why `quarto_reveal`'s Dockerfile gives Poetry a separate venv under `POETRY_HOME`). Since Poetry 2 it uses the standard `[project]` table.

```
poetry new myproj # or `poetry init` in an existing folder
poetry add pandas "numpy<2"
poetry add --group dev pytest
poetry remove pandas
poetry lock # re-resolve after editing pyproject.toml (keeps versions where possible)
poetry lock --regenerate # re-resolve from scratch
poetry update numpy # newest allowed version of one package
poetry install # create/update the venv from the lock
poetry install --no-root # dependencies only (Docker, or projects that are not packages)
poetry sync # like install, and also remove what is not in the lock
poetry show --tree # dependency tree; poetry show --outdated
poetry run python x.py
```

Poetry's environments

Poetry names environments itself: `<project>-<hash>-py3.12`, stored in `~/.cache/pypoetry/virtualenvs/`. You cannot choose the name. Per project, the only variation is the Python version:

```
poetry env use 3.11          # create/select myproj-Ab3xQ-py3.11
poetry env use 3.12          # create/select myproj-Ab3xQ-py3.12
poetry env list              # the active one is marked (Activated)
poetry env info --path
poetry env remove python3.11
eval $(poetry env activate) # activate in the shell (Poetry 2; `poetry shell` is now a plugin)
poetry config virtualenvs.in-project true --local # use ./venv instead (writes poetry.toml)
```

A different *set of packages* means a different project folder, as with `uv. poetry export` (to `requirements.txt`) is no longer built in: it needs `poetry self add poetry-plugin-export`.

7 · conda, mamba and micromamba

Three programs for the same package format and the same channels:

Tool	What it is	When to use
conda	The original, written in Python. Used to be very slow at solving; now uses mamba's solver (libmamba).	If an installation already has it (Anaconda, Miniforge).
mamba	Fast C++ reimplementation with the same commands.	Desktop use via Miniforge.
micromamba	mamba as one static executable: no Python, no "base" environment to look after.	Docker and CI (the learning containers), or a minimal personal install.

Channels are the repositories. **conda-forge** is the community channel (~25k packages: Sage, graph-tool, xeus-cpp, R and CRAN packages, GDAL, compilers). Always name it explicitly (`-c conda-forge`). Anaconda's own `defaults` channel has terms of service that can require a paid licence for larger organisations; conda-forge is free and community-run. Miniforge and micromamba use conda-forge by default or when told.

The intent file: `environment.yml`

```
name: gt
channels: [conda-forge]
dependencies:
  - python=3.12
  - graph-tool>=2.98
  - pandas
  - pip
  - pip:          # PyPI-only extras, installed by pip after the conda packages
    - infomap
```

Making and reading the lists

```
micromamba env export -n gt --from-history > environment.yml # only what you explicitly asked for
micromamba env export -n gt > env-full.yml # everything, with versions and builds
micromamba env export -n gt --explicit > gt-linux-64.lock # exact package URLs (+md5): a true lock
micromamba list -n gt # what is installed
```

conda-lock (a separate tool) turns `environment.yml` into one `conda-lock.yml` covering several platforms: `conda-lock -f environment.yml -p linux-64 -p osx-arm64`. micromamba installs such a file directly. An `--explicit` lock is for one platform only.

Managing environments

```
micromamba create -n gt -f environment.yml          # named: lives in $MAMBA_ROOT_PREFIX/envs/gt
micromamba create -p /opt/gt -f environment.yml      # at a path (what the Dockerfiles do)
micromamba create -n gt -c conda-forge python=3.12 graph-tool # without a file
micromamba create -n gt -f gt-linux-64.lock         # rebuild exactly from a lock
micromamba install -n gt -c conda-forge networkx    # add a package
micromamba update -n gt --all                       # update everything allowed
micromamba env update -n gt -f environment.yml --prune # bring env in line with the file
micromamba env list
micromamba env remove -n gt
micromamba run -n gt python script.py              # run without activating
eval "$(micromamba shell hook -s bash)"             # once per shell (or `micromamba shell init -s
bash`)
micromamba activate gt; micromamba deactivate
```

Replace `micromamba` with `conda` or `mamba` for the other two; the main difference is that `conda`'s file-based `create` is `conda env create -f environment.yml`.

Two classic traps. (1) `micromamba install x` does *not* add `x` to `environment.yml`: edit the file and recreate, or re-export with `--from-history`. (2) `pip install` inside a `conda` environment can silently replace a package `conda` installed (`numpy` above all). Install everything you can from `conda-forge` first, put the PyPI-only rest in the `pip:` section, and avoid running `conda` after `pip` in the same environment.

8 · Other tools worth knowing

- **pixi** (from the `mamba` developers): `uv`-style project management for `conda-forge` packages: `pixi init`, `pixi add graph-tool`, `pixi.lock`, `pixi run`. Unlike `uv` and `Poetry` it supports several *named environments inside one project* (built from "features"). It is the natural upgrade if you want locked `conda` environments without `conda-lock`.
- **Spack**: builds everything from source with exact compilers, MPI flavours and build options. The standard on HPC clusters; heavy and slow for a single container.
- **Nix / Guix**: functional package managers that make whole systems bit-for-bit reproducible. Powerful, with a steep learning curve.
- **Docker** is not a package manager but a way to freeze the *result*: an image is a whole environment, OS included. Inside it you still use `apt/uv/micromamba`, and lock files are still what make a rebuild of the image identical.

9 · Jupyter kernels: one front end, many environments

When you run a cell, the front end (JupyterLab, Quarto, `jupyter console`, an editor plugin) does not execute the code. It starts a separate program, the **kernel**, and sends it the code over local sockets (ZeroMQ); the kernel runs it and sends back output, errors and plots. **ipykernel** is the package that turns a Python interpreter into such a kernel. Two consequences:

- `ipykernel` must be installed in each environment whose packages you want to use from Jupyter.
- The front end can live in a *different* environment. In the learning containers, JupyterLab and Quarto run from `/opt/venv`; the kernels they start run `/opt/thesis/bin/python`, `/opt/gt3/bin/python`, Sage, Julia, R...

A kernelspec is just a small JSON file

Jupyter finds kernels by looking for folders `kernels/<name>/kernel.json` in its data directories. The folder name is the kernel's ID (what `jupyter: thesis` in a `.qmd` refers to). The file says which program to start:

```
{ "argv": ["/opt/thesis/bin/python", "-m", "ipykernel_launcher", "-f", "{connection_file}"],
  "display_name": "Python 3.12 (thesis · graph-tool)",      ← label in the kernel picker
  "language": "python",                                  ← Quarto matches {python} cells to this
  "env": { "PATH": "/some/env/bin:..." } }              ← optional: variables set for the kernel
```

{connection_file} is a placeholder: at start-up Jupyter replaces it with a small JSON file holding the ports and the secret key for that session. You can edit kernel.json by hand: rename it, add an `env` block, or point `argv` at another interpreter.

Writing one: `python -m ipykernel install`

This installs no software; it only writes the kernel.json. The interpreter written into `argv` is **the Python that runs the command**, so always start the command with the environment's own Python.

Option	Effect
<code>--name thesis</code>	folder name = kernel ID. Default: <code>python3</code>
<code>--display-name "..."</code>	label in the picker. Default: the <code>--name</code> if given, otherwise "Python 3 (ipykernel)"
<code>--user</code>	writes to <code>~/.local/share/jupyter/kernels/<name>/</code>
<code>--prefix /usr/local</code>	writes to <code>/usr/local/share/jupyter/kernels/<name>/</code>
<code>--sys-prefix</code>	writes into the environment running the command (<code><env>/share/jupyter/kernels/</code>)
<code>--env NAME VALUE</code>	adds an <code>env</code> entry (repeatable)

```
# register any environment so JupyterLab and Quarto can use it:
~/envs/stats/.venv/bin/python -m ipykernel install --user --name stats --display-name "Python (stats)"
micromamba run -n gt python -m ipykernel install --user --name gt --display-name "Python (graph-tool, conda)"
jupyter kernelspec list          # every kernel and its folder
jupyter kernelspec remove stats  # delete one
jupyter --paths                  # the folders searched, in order
```

Search order and the default `python3`

If two folders contain a kernel with the same name, Jupyter uses the first one found. When Jupyter itself runs from a `venv` or `conda` environment (the normal case, and the case in the containers), the order is: **that environment's** `share/jupyter`, then the user folder `~/.local/share/jupyter`, then the system folders `/usr/local/share/jupyter` and `/usr/share/jupyter`. (Setting `JUPYTER_PREFER_ENV_PATH=0` puts the user folder first.)

`python3` is the kernel used when a notebook does not name one. Installing `ipykernel` into an environment writes a `python3` kernelspec for that environment, so you can redirect the default by overwriting that file, which is what the thesis image does (next section).

10 · Worked example: the thesis image

`~/dockerfiles/base_learning/thesis/` needs `graph-tool` 2.59 (the version the lessons' numbers were checked with) and `graph-tool` 3.x (the new API). They cannot share an environment: Ubuntu's 2.59 is compiled against `numpy` 1.26, `conda-forge`'s 3.x against `numpy` 2. So the image has three Python environments:

```

/opt/venv    uv venv, numpy 2          the front end: JupyterLab, Jupyter server, Quarto's Python
                                         (your thesis code does not run here)

/opt/thesis  venv --system-site-packages on Ubuntu's python3.12
              sees apt's graph-tool 2.59 + numpy 1.26 + scipy/pandas/sklearn...
              pip pinned by constraints.txt; infomaps from PyPI
              → kernels "thesis" and "python3"; `python`, `ipython`, `pip` in a shell

/opt/gt3     micromamba prefix from conda-forge: own python, numpy 2, graph-tool 3.x
              → kernel "graph-tool-3"; `ipython-gt3` in a shell

```

How the kernels are registered

```

# graph-tool 2.59: two names, same interpreter
/opt/thesis/bin/python -m ipykernel install --prefix /usr/local \
    --name thesis --display-name "Python 3.12 (thesis · graph-tool)"
/opt/thesis/bin/python -m ipykernel install --prefix /opt/venv \
    --name python3 --display-name "Python 3.12 (thesis · graph-tool)" # replaces /opt/venv's default

# graph-tool 3: written inside the prefix, then copied where /opt/venv's Jupyter looks
/opt/gt3/bin/python -m ipykernel install --prefix /opt/gt3 \
    --name graph-tool-3 --display-name "Python (graph-tool 3, conda-forge)"
register-conda-kernels /opt/gt3 --only graph-tool-3

```

register-conda-kernels (in `base_learning/bin/`) copies `<prefix>/share/jupyter/kernels/<name>/` to `/usr/local/share/jupyter/kernels/` and adds an `env` block with `PATH=<prefix>/bin:...` and `CONDA_PREFIX`, i.e. what `micromamba activate` would have done. The programming image uses it the same way for the `xeus-cpp` C++ kernels. Each build step starts its kernel and imports `graph-tool`, so a broken kernel fails the image build instead of failing later in a lecture.

Choosing an environment when you work

Where	graph-tool 2.59	graph-tool 3
.qmd front matter	jupyter: thesis, or no jupyter: line	jupyter: graph-tool-3
JupyterLab	"Python 3.12 (thesis · graph-tool)"	"Python (graph-tool 3, conda-forge)"
Terminal	python, ipython	ipython-gt3, /opt/gt3/bin/python
Add a package	pip install x (obeys constraints.txt)	micromamba install -p /opt/gt3 -c conda-forge x

The terminal needs one extra trick. The image sets `PATH` and `VIRTUAL_ENV` to `/opt/thesis`, so a bare `python` is the thesis one. IPython reads `VIRTUAL_ENV` at start-up and puts that `venv`'s packages first on `sys.path`, so `ipython-gt3` runs `env -u VIRTUAL_ENV /opt/gt3/bin/python -m IPython`; otherwise `graph-tool 3` would pick up `numpy 1.26` and fail. Kernels skip this IPython check, which is why the bug only showed up in the terminal.

Where else conda is used in the containers

Only where nothing else works: SageMath in `math` (`/opt/sage`; the install route the Sage docs recommend), `xeus-cpp` in `programming` (`/opt/xeus-cpp`; built against a specific LLVM, not on apt or PyPI), and `graph-tool 3` in `thesis`. Everything else is apt plus the uv-managed `/opt/venv`. These three prefixes are currently **unpinned** (newest unless the version ARG is set): adding `--explicit` lock files, or moving to `pixi`, would make rebuilds reproducible.

11 · Which tool when, and common pitfalls

Need	Use
A code project (portal, app, package, analysis repo)	uv project mode; commit <code>pyproject.toml</code> + <code>uv.lock</code>
A project that already uses Poetry	Poetry : <code>poetry install</code> ; migrate only if it bothers you
A shared venv in a container (<code>/opt/venv</code>)	uv pip mode, ideally from a compiled <code>requirements.txt</code>
OS tools and libraries; a package Ubuntu builds well	apt
Big, mostly non-Python stacks (Sage, graph-tool, GDAL stacks, R + Python together, C++ kernels)	micromamba (or <code>pixi</code>), one prefix each, conda-forge only
Scratch environments used from any folder	micromamba named environments, or uv Option B (<code>~/.venvs</code> + <code>workon</code>)
HPC cluster software with specific compilers / MPI	The cluster's modules or Spack

Pitfalls, and how to recognise them

Symptom	Cause and fix
"compiled using NumPy 1.x cannot be run in NumPy 2.x"	Two numpy versions met in one process: a compiled package from one environment, numpy from another. Keep such packages in separate environments; pin numpy with a constraints file.
Kernel starts but <code>import x</code> fails although <code>x</code> is installed	The kernel points at a different interpreter. <code>cat</code> its <code>kernel.json</code> ; re-run <code>ipykernel install</code> with the right environment's Python.
Terminal IPython sees the wrong packages	A stale <code>VIRTUAL_ENV</code> (IPython honours it). Unset it, or set it to the environment you mean.
A conda environment broke after <code>pip install</code>	pip replaced a conda package. Recreate from the lock; install from conda-forge first, pip last.
The rebuild gives different versions	No lock file, or building from the intent file. Build from the lock (<code>uv sync --locked</code> , <code>poetry install</code> , <code>micromamba create -f <lock></code>).
<code>conda activate</code> in a Dockerfile "does nothing"	Each <code>RUN</code> is a new shell. Call <code><prefix>/bin/python</code> directly, use <code>micromamba run -p</code> , or set <code>PATH</code> with <code>ENV</code> .
uv / Poetry refuse an older Python	<code>requires-python</code> in <code>pyproject.toml</code> excludes it; lower it.
Root-owned files in a container's home or venv	Packages installed as root during the build; <code>chown</code> the environment to the user in the same layer (see the <code>base_learning</code> Dockerfiles).

12 · Quick reference

Task	venv + pip	uv (project)	Poetry	micromamba
Create	<code>python -m venv .venv</code>	<code>uv init → uv sync</code>	<code>poetry new/init → poetry install</code>	<code>create -n NAME -f env.yml</code>
Add package	<code>pip install x</code>	<code>uv add x</code>	<code>poetry add x</code>	<code>install -n NAME -c conda-forge x</code>
Dev-only package	—	<code>uv add --dev x</code>	<code>poetry add --group dev x</code>	separate env
Remove package	<code>pip uninstall x</code>	<code>uv remove x</code>	<code>poetry remove x</code>	<code>remove -n NAME x</code>
Lock	<code>pip freeze > req.txt</code>	<code>uv lock</code>	<code>poetry lock</code>	<code>env export --explicit</code>
Upgrade one	<code>pip install -U x</code>	<code>uv lock --upgrade-package x</code>	<code>poetry update x</code>	<code>update -n NAME x</code>
Rebuild exactly	<code>pip install -r req.txt</code>	<code>uv sync --locked</code>	<code>poetry sync</code>	<code>create -n NAME -f LOCK</code>
List / tree	<code>pip list</code>	<code>uv tree</code>	<code>poetry show --tree</code>	<code>list -n NAME</code>
Run without activating	<code>.venv/bin/python x.py</code>	<code>uv run x.py</code>	<code>poetry run python x.py</code>	<code>run -n NAME python x.py</code>
Activate	<code>source .venv/bin/activate</code>	same	<code>eval \$(poetry env activate)</code>	<code>micromamba activate NAME</code>
List environments	<code>ls ~/.venvs</code>	— (one per project)	<code>poetry env list</code>	<code>env list</code>
Delete environment	<code>rm -rf .venv</code>	<code>rm -rf .venv</code>	<code>poetry env remove PY</code>	<code>env remove -n NAME</code>
Jupyter kernel	<code><env's python> -m ipykernel install --user --name NAME --display-name "LABEL"</code> (ipykernel must be installed in that environment)			

micromamba commands are written without the leading `micromamba`; `-n NAME` can always be replaced by `-p /path/to/prefix`.